

Pre requisites

- Python 3.10+ installed
- pip install -U google-genai
- A free Gemini API key from Google AI Studio
- A code editor (VS Code recommended)
- Test snippet to confirm setup works before arriving:

Topic 1: Theoretical Explanations Covered

1. What "Autonomous AI" Actually Means

- **Chatbot vs. Autonomous Agent:** A chatbot maps a single input to a single output within one call, waiting for user input and forgetting previous state unless re-prompted. An autonomous agent wraps the language model in an execution loop, allowing it to take multiple actions to achieve a high-level goal, dynamically decide its sequence of steps, use tools, evaluate its own progress, and adjust without constant human intervention.
- **Core Definition of Autonomy:** Autonomy refers specifically to **who decides the number and sequence of steps**. In a chatbot, step count is fixed at one; in an agent, the model determines its execution path dynamically based on observations within the run.
- **The ReAct Pattern:** This dynamic reasoning loop is formalized in AI literature as **ReAct (Reason + Act)**. The agent alternates between reasoning about what to do next, taking an action (such as executing code or invoking an API), observing the result, and reasoning again.
 - **Thought:** The AI looks at the problem and types out its plan (e.g., *"I don't know the weather today, so I should check a weather app."*)
 - **Action:** The AI actually uses a tool (e.g., it clicks a button or runs a quick search to check the weather).
 - **Observation:** The AI looks at the result it got back (e.g., *"The app says it is raining."*) and uses that new information to take its next step.

autonomy = the model deciding the next action itself, not you scripting each step.

Technical Terms

Long-term memory — staying consistent across many runs.

- Mental model: a notebook the agent reads before acting and writes to after.
- Example: an agent that remembers "this teacher prefers short summaries" from last week's interaction and applies it again today, without being told again.
- Practical distinction to make explicit: this is different from the *context window* (what's in the current conversation) — memory persists *between* separate runs, context is what's visible *within* one run.

Context management — making sure the agent only sees what's relevant.

- Mental model: you don't hand someone your entire filing cabinet to answer one question — you hand them the right folder.
- Example: a supervisor agent reviewing a draft doesn't need the entire chat history of how the draft was written — it needs the plan and the draft, nothing else. Too much irrelevant context degrades decision quality and wastes tokens.

Decision-making — how the agent picks its next move.

- Mental model: the agent loop is really just: *read the goal + context* → *decide: answer directly, or call a tool?* → *act* → *look at the result* → *repeat until done*.
- Show this as a simple loop diagram on the board:

2. The 5-Step Decision Loop

The core execution engine of an autonomous agent follows a 5-step loop:

1. **Read goal + context:** Inspect the assigned objective and current state.
 2. **Decide (Answer or Tool?):** Determine whether to return a direct answer or invoke an external tool.
 3. **Act:** Execute the chosen action or tool call.
 4. **Observe result:** Receive and inspect the output produced by the action.
 5. **Check completion:** If done, terminate and output the result; if not, loop back to Step 1.
- **Exit Conditions:** Every agent loop requires an explicit termination check or iteration cap; otherwise, unexpected tool outputs or model revisions risk creating infinite execution loops.

3. Execution & Triggering Models

Agents are initiated using two main execution patterns:

- **Polling / Scheduled Triggers:** The agent wakes up on a set timer (e.g., cron jobs, `time.sleep` loops) to check a condition and act if needed. This is cheap and requires zero external infrastructure, but incurs latency and wastes checks if events are rare.
- **Event-Driven Triggers:** The agent is triggered immediately by an external event (e.g., webhooks, form submissions, new database rows). This provides real-time responsiveness and efficiency, but depends on external systems supporting callbacks.

4. Long-Term Memory vs. Context Window

Distinguishing between persistent memory and the context window is critical:

- **Context Window:** The information visible **within a single execution run** (system prompt, conversation history, pasted documents). Context exists only for the duration of one API call or session and vanishes unless saved externally.
- **Long-Term Memory:** Persistent storage (e.g., JSON files, databases, vector indices) written to specifically so that **future, separate runs** can retrieve past state.

5. Context Management & Scaling Techniques

Every model has a finite context window token limit. Even well within token limits, passing irrelevant context increases token costs, distracts the model with noise, and degrades decision quality. The goal of context management is to hand the agent "the specific folder" rather than "the entire filing cabinet".

- **Manual Scoping:** The developer explicitly restricts each agent's input string to only what its specific role requires (e.g., passing only the outline and draft to a supervisor agent). This is the simplest and most reliable approach for structured pipelines.
- **Windowing:** Retaining only the last N turns of a conversation and dropping older turns.
- **Summarization:** Periodically compressing long execution histories into short summary paragraphs.
- **Retrieval (RAG):** Storing information externally and retrieving only relevant chunks dynamically based on the current step.

6. The 4 Pillars of Agent Operations

Agent lifecycle management is divided across four distinct operational phases based on when they occur relative to execution:

Concept	Execution Timing	Primary Question Addressed
Monitoring	<i>During</i> the run	"Is the agent executing correctly or stuck in a loop right now?"
Logging	<i>During</i> , read <i>after</i>	"What exact steps and inputs occurred, so we can audit or reconstruct the run?"
Evaluating	<i>After</i> the run	"Was the final output high quality and compliant with requirements?"
Maintaining	<i>Ongoing / Between</i> runs	"Do system prompts, tools, or model configs need updates due to shifting requirements?"

What we will build today:

Project: Three agents collaborate to produce a 3-day GenAI workshop syllabus.

1. **Planner agent** — drafts a 3-day outline (topics per day, no full content).
2. **Worker agent** — takes the outline and writes full content for each day.
3. **Supervisor agent** — reviews the worker's draft against the planner's outline, adds comments, and either approves it (ready to email to a manager) or sends it back with revision notes.

VERY IMPORTANT (when re-teaching)

- The chatbot-vs-agent contrast
- The five-step decision loop
- The four operations concepts (monitor / log / evaluate / maintain), even at a plain-language level

Topic 2: Coding Best Practices

1. API Security & Environment Configuration

- **Environment Files (`python-dotenv`):** Store API credentials in a local `.env` file and load them via `dotenv.load_dotenv()` prior to instantiating API clients rather than manually exporting keys in shell sessions.
- **Version Control Hygiene:** Always list `.env` in `.gitignore` to prevent committing secrets to code repositories.
- **Centralized Constants:** Pull global configuration parameters (such as `MODEL = "gemini-2.5-flash"` and `MAX_RETRIES = 3`) to the top of the module as named constants.

2. Type Safety & Thin Function Wrappers

Dedicated Agent Signatures: Rather than calling generic API dispatch functions directly with raw strings, wrap each agent in a thin, typed Python function:

```
def run_planner(topic: str) -> str:  
    return call_agent(planner_prompt, f"Workshop topic: {topic}")
```

```
def run_worker(outline: str) -> str:  
    return call_agent(worker_prompt, f"Outline:\n{outline}")
```

```
def run_supervisor(outline: str, draft: str) -> SupervisorReview:  
    return call_agent(supervisor_prompt, f"OUTLINE:\n{outline}\n\nDRAFT:\n{draft}")
```

-
- **Self-Documenting Context:** Function signatures serve as explicit code documentation enforcing exactly what context inputs each agent requires.

3. Fault Tolerance & Retries with Exponential Backoff

- **API Exception Handling:** Intercept API error exceptions (e.g., `google.genai.errors.APIError`) to prevent transient rate limits or network disruptions from crashing multi-step pipelines.

Exponential Backoff: Implement backoff logic (`time.sleep(2 ** attempt)`) within retry loops:
import time

```
from google.genai.errors import APIError
```

```
def call_agent(system_prompt: str, user_input: str, max_retries: int = 3) -> str:
    prompt = f"{system_prompt}\n\n---\n\n{user_input}"
    for attempt in range(1, max_retries + 1):
        try:
            response = client.models.generate_content(model=MODEL, contents=prompt)
            return response.text
        except APIError as e:
            if attempt == max_retries:
                raise
            wait = 2 ** attempt
            print(f"API error ({e}), retrying in {wait}s...")
            time.sleep(wait)
```

-
- **Iteration Caps on Re-loops:** When implementing automated revision loops between agents (e.g., Worker re-drafting upon Supervisor rejection), enforce a hard maximum attempt threshold (e.g., capping at 3 attempts) to avoid infinite loops and API token exhaustion.

4. Typed Structured Output via Pydantic

- **Eliminating Substring Parsing:** Checking free-text responses using string matching (e.g., `if "DECISION: APPROVED" in review:`) is fragile due to casing variations, extra whitespace, or false positives where the model discusses approval while rejecting a draft.

Schema Definition with Pydantic & Enums: Define rigid output structures using Pydantic `BaseModel` and `str`, `Enum` classes:

```
from enum import Enum
```

```
from pydantic import BaseModel
```

```
class Decision(str, Enum):
    APPROVED = "APPROVED"
    REVISE = "REVISE"
```

```
class SupervisorReview(BaseModel):
    comments: str
    decision: Decision
```

-
- **Enforcing SDK Schemas:** Pass `response_mime_type="application/json"` and `response_schema=SupervisorReview` inside `types.GenerateContentConfig`.
- **Prompt Simplification:** With structured schemas, system prompts no longer need formatting instructions; they can focus entirely on the core evaluation task.

Handling Null Returns: Because SDKs may suppress validation errors and return `response.parsed` as `None`, explicitly validate output presence:

```
review: SupervisorReview | None = response.parsed
```

if review is None:

```
    raise ValueError("Supervisor response did not match expected schema")
```

-

Type-Safe Control Flow: Downstream code can use pattern matching or enum inspections to branch execution reliably:

match review.decision:

```
    case Decision.APPROVED:
```

```
        print(">> Pipeline complete. Ready for distribution.")
```

```
    case Decision.REVISE:
```

```
        print(">> Sent back for revision.")
```

-

- **Rule of Thumb:** Reserve structured schema outputs for instances where **downstream code must branch or act on specific fields**, while keeping free-form content (like outline prose or session text) as standard strings.

5. Testing & Logging Hygiene

- **Mocking API Calls for Testing:** Use `unittest.mock.patch` to mock model responses, allowing developers to test multi-agent branching logic without consuming API quota.
- **Structured Logging over Stdout:** Replace standard `print()` statements with Python's native `logging` module so full execution traces can be saved to log files for auditing, post-run evaluation, and debugging.

💡 Would you like to review a complete, production-ready Python script that implements the entire 3-agent pipeline using Pydantic structured output, exponential backoff retries, and logging?